Course Code: CA-414
Object Oriented Programming using C++

By

Rakesh Yashwant Gedam

(Assitant Professor School of Computer science, KBCNMU Jalgaon)

What is C?

- C is a structural or procedural oriented programming language which is machine-independent and extensively used in various applications.
- C is the basic programming language that can be used to develop from the operating systems (like Windows) to complex programs like Oracle database, Git, Python interpreter, and many more. C programming language can be called a god's programming language as it forms the base for other programming languages.
- If we know the C language, then we can easily learn other programming languages.
- C language was developed by the great computer scientist Dennis Ritchie at the Bell Laboratories. It contains some additional features that make it unique from other programming languages.

What is C++?

- C++ is a special-purpose programming language developed by **Bjarne Stroustrup** at Bell Labs circa 1980.
- C++ language is very similar to C language, and it is so compatible with C that it can run 99% of C programs without changing any source of code though C++ is an object-oriented programming language, so it is safer and well-structured programming language than C.

differences between C and C++.

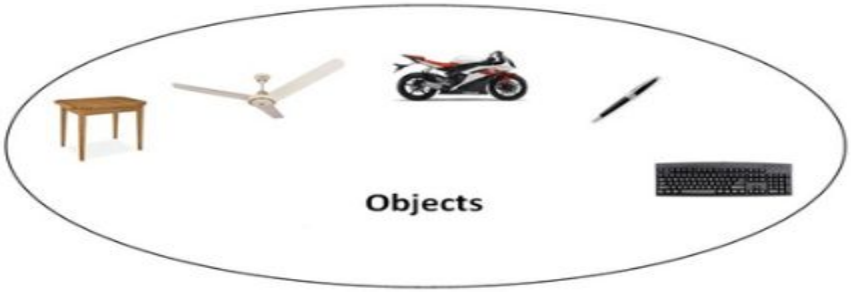
No.	C	C++
1)	C follows the **procedural style programming.**	C++ is multi-paradigm. It supports both **procedural and object oriented.**
2)	Data is less secured in C.	In C++, you can use modifiers for class members to make it inaccessible for outside users.
3)	C follows the **top-down approach.**	C++ follows the **bottom-up approach.**
4)	C does not support function overloading.	C++ supports function overloading.
5)	In C, you can't use functions in structure.	In C++, you can use functions in structure.
6)	C does not support reference variables.	C++ supports reference variables.
7)	In C, **scanf() and printf()** are mainly used for input/output.	C++ mainly uses stream **cin and cout** to perform input and output operations.
8)	Operator overloading is not possible in C.	Operator overloading is possible in C++.
9)	C programs are divided into **procedures and modules**	C++ programs are divided into **functions and classes.**
10)	C does not provide the feature of namespace.	C++ supports the feature of namespace.
11)	Exception handling is not easy in C. It has to perform using other functions.	C++ provides exception handling using Try and Catch block.
12)	C does not support the inheritance.	C++ supports inheritance.

OOPs
(Object Oriented Programming)

- **Object** means a real word entity such as pen, chair, table etc.
- **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects.
- It simplifies the software development and maintenance by providing some concepts:
  - Object
  - Class
  - Inheritance
  - Polymorphism
  - Abstraction
  - Encapsulation

Object

- Objects are key to understanding *object-oriented* technology.
- Any entity that has state and behavior is known as an object.
- You can look around you now and see many examples of real-world objects.
- Example your dog, your desk, your television set, your bicycle.T
- hese real-world objects share two characteristics:
- They all have *state* and *behavior*.
- For example, dogs have state (name, color, breed) and behavior (barking, eating, and wagging tail).
- Bicycles have state (current gear, current pedal cadence, two wheels, number of gears) and behavior (braking, accelerating, slowing down, changing gears).
- For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.



Class

- A Class in C++ is the foundational element that leads to Object-Oriented programming.
- **Collection of objects** is called class.
- It is a logical entity.
- A class instance must be created in order to access and use the user-defined data type's data members and member functions.
- An class acts as its blueprint.
- Take the class of cars as an example.
- Even if different names and brands may be used for different cars, all of them will have some characteristics in common, such as four wheels, a speed limit, a range of miles, etc.
- In this case, the class of car is represented by the wheels, the speed limitations, and the mileage.

Inheritance

- **When one object acquires all the properties and behaviours of parent object** i.e. known as inheritance.
- It provides code reusability.
- It is used to achieve runtime polymorphism.
- Sub class - Subclass or Derived Class refers to a class that receives properties from another class.
- Super class - The term "Base Class" or "Super Class" refers to the class from which a subclass inherits its properties.
- Reusability - As a result, when we wish to create a new class, but an existing class already contains some of the code we need, we can generate our new class from the old class thanks to inheritance. This allows us to utilize the fields and methods of the pre-existing class.

Polymorphism

- When **one task is performed by different ways** i.e. known as polymorphism.
- For example: Mobile keypad used for calling and test msg
- Different situations may cause an operation to behave differently.
- The type of data utilized in the operation determines the behavior.

Abstraction

- **Hiding internal details and showing functionality** is known as abstraction.
- Data abstraction is the process of exposing to the outside world only the information that is absolutely necessary while concealing implementation or background information.
- For example: phone call, we don't know the internal processing.
- In C++, we use abstract class and interface to achieve abstraction.

POP vs OOP's

Procedural Programming Language	Object Oriented Programming Language
1. Program is divided into functions.	1. Program is divide into classes and objects..
2. The emphasis is on doing things.	2. The emphasis on data.
3. Poor modeling to real world problems.	3. Strong modeling to real world problems.
4. It is not easy to maintain project if it is too complex.	4. It is easy to maintain project even if it is too complex.
5. Provides poor data security.	5. Provides strong data Security.
6. It is not extensible programming language.	6. It is highly extensible programming language.
7. Productivity is low.	7. Productivity is high.
8. Do not provide any support for new data types.	8. Provide support to new Data types.
9. Unit of programming is function.	9. Unit of programming is class.
10. Ex. Pascal , C , Basic , Fortran.	10. Ex. C++ , Java , Oracle.

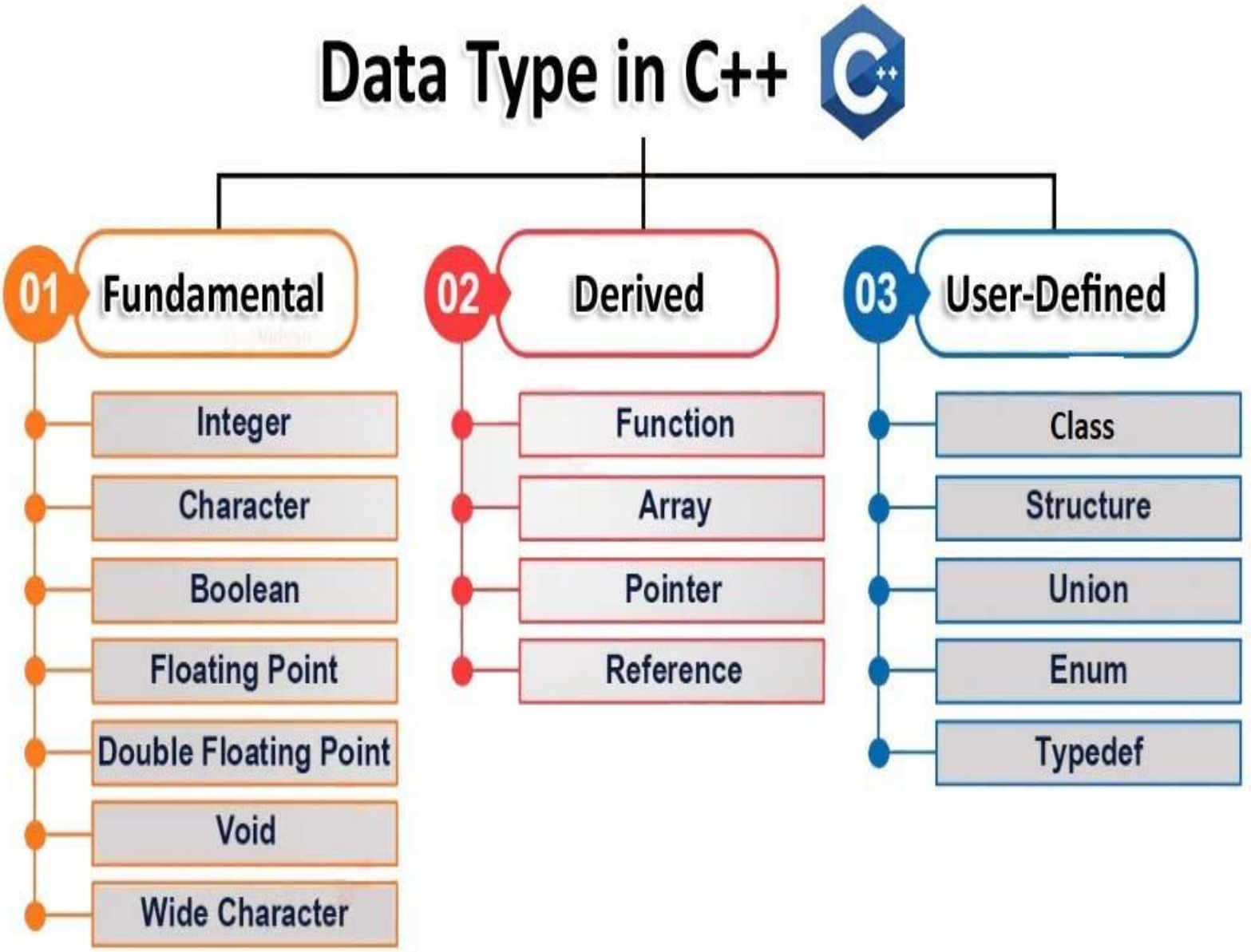
Variable

- ***Variables*** are the fundamental building blocks of data manipulation and storage in programming, acting as ***dynamic containers*** for data in the C++ programming language.
- A ***variable*** is more than just a memory label. It serves as a link between abstract ideas and concrete data storage, allowing programmers to deftly manipulate data.
- With the help of C++ variables, developers may complete a wide range of jobs, from simple ***arithmetic operations*** to complex algorithmic designs.
- These programmable containers can take on a variety of shapes, such as **inte*gers,* *floating-point numbers, characters***, and ***user-defined structures***, each of which has a distinctive impact on the operation of the program.
- Programmers follow a set of guidelines when generating variables, creating names that combine alphanumeric letters and underscores while avoiding reserved keywords. More than just placeholders, variables are what drive ***user input, intermediary calculations***, and the dynamic interactions that shape the program environment.
- A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.
- Example :

```
int x=5,b=10;  //declaring 2 variable of integer type
float f=30.8;
char c='A';
```

C++ Data Types

- *Data type tells what kind of data is going to store for variable .*
- All variables use data type during declaration to restrict the type of data to be stored.
- Therefore, we can say that data types are used to tell the variables the type of data they can store.
- Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data type with which it is declared.
- Every data type requires a different amount of memory.
- C++ supports a wide variety of data types and the programmer can select the data type appropriate to the needs of the application.
- Data types specify the size and types of values to be stored. However, storage representation and machine instructions to manipulate each data type differ from machine to machine, although C++ instructions are identical on all machines.


Data Type in C++
01
Fundamental
Integer
Character
Boolean
Floating Point
Double Floating Point
Void
Wide Character
02
Derived
Function
Array
Pointer
Reference
03
User-Defined
Class
Structure
Union
Enum
Typedef

Primitive Data Types

- **Integer**: The keyword used for integer data types is **int**. Integers typically require 4 bytes of memory space and range from -2147483648 to 2147483647.
- **Character**: Character data type is used for storing characters. The keyword used for the character data type is **char**. Characters typically require 1 byte of memory space and range from -128 to 127 or 0 to 255.
- **Boolean**: Boolean data type is used for storing Boolean or logical values. A Boolean variable can store either *true* or *false*. The keyword used for the Boolean data type is **bool**.
- **Floating Point**: Floating Point data type is used for storing single-precision floating-point values or decimal values. The keyword used for the floating-point data type is **float**. Float variables typically require 4 bytes of memory space.
- **Double Floating Point**: Double Floating Point data type is used for storing double-precision floating-point values or decimal values. The keyword used for the double floating-point data type is **double**. Double variables typically require 8 bytes of memory space.
- **void**: Void means without any value. void data type represents a valueless entity. A void data type is used for those function which does not return a value.
- **Wide Character**: Wide character data type is also a character data type but this data type has a size greater than the normal 8-bit data type. Represented by **wchar_t**. It is generally 2 or 4 bytes long.
- **sizeof() operator:** sizeof() operator is used to find the number of bytes occupied by a variable/data type in computer memory.

```
// C++ Program to Demonstrate the correct size of various
   data types on your computer.
#include <iostream>
using namespace std;
int main()
{
   cout << "Size of char : " << sizeof(char) << endl;
   cout << "Size of int : " << sizeof(int) << endl;
   cout << "Size of long : " << sizeof(long) << endl;
   cout << "Size of float : " << sizeof(float) << endl;
   cout << "Size of double : " << sizeof(double) << endl;
   return 0;
}
```

Derived Data Types:

- Derived data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types.
- These can be of four types namely:
  - ❖ Function
  - ❖ Array
  - ❖ Pointer
  - ❖ Reference

Functions

- The function in C++ language is also known as procedure or subroutine in other programming languages.
- To perform any task, we can create function. A function can be called many times.
- It provides modularity and code reusability.

Functions

- A function is a group of statements that together perform a task. Every C++ program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.
- You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division usually is such that each function performs a specific task.
- A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.

There are two types of function:

- **Standard Library Functions:** Predefined in C++
- **User-defined Function:** Created by users

Function Declarations

- A function **declaration** tells the compiler about a function name and how to call the function.
- The actual body of the function can be defined separately.

- *return_type function_name(parameter list);*

Defining a Function

```
return_type function_name( parameter list )
{
body of the function
}
```

- **Return Type** − A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword **void**.
- **Function Name** − This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- **Parameters** − A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- **Function Body** − The function body contains a collection of statements that define what the function does.

Calling a Function

- While creating a C++ function, you give a definition of what the function has to do. To use a function, you will have to call or invoke that function.
- When a program calls a function, program control is transferred to the called function. A called function performs defined task and when it's return statement is executed or when its function-ending closing brace is reached, it returns program control back to the main program.
- To call a function, you simply need to pass the required parameters along with function name, and if function returns a value, then you can store returned value

- // **Function declaration**
  void myFunction();

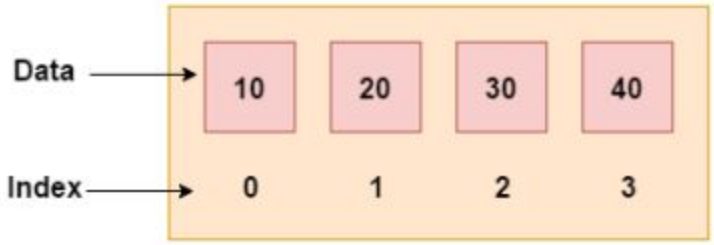
  // The main method
  int main() {
  myFunction(); // **call** the function
  return 0;
  }

  // **Function definition**
  void myFunction() {
  cout << “Hellow , Welcome our class!";
  }

```
// using function definition after main() function , function prototype is declared before
      main()

#include <iostream>
using namespace std;
int add(int, int); // function prototype or function declaration
int main()
 {
    int sum;
   sum = add(100, 78); // calling the function and storing the returned value in sum
    cout << "100 + 78 = " << sum << endl;
    return 0;
}
int add(int a, int b) // function definition
{
    return (a + b);
}
```

Arrays

- Like other programming languages, array in C++ is a group of similar types of elements that have contiguous memory location.
- In C++ **std::array** is a container that encapsulates fixed size arrays.
- In C++, array index starts from 0.
- We can store only fixed set of elements in C++ array.
- A collection of related data items stored in adjacent memory places is referred to as an array in the C/C++ programming language or any other programming language for that matter.



Example

```
#include <iostream>
using namespace std;
int main()
{
  int myNumbers[5] = {10, 20, 30, 40, 50};
  for (int i = 0; i < 5; i++)
     {
       cout << myNumbers[i] << "\n";
     }
  return 0;
}
```

Structure

```
#include <iostream>
#include <string>
using namespace std;

int main() {
  struct {
    string brand;
    string model;
    int year;
  } myCar1, myCar2; // We can add variables by separating them with a comma here

  // Put data into the first structure
  myCar1.brand = "Maruti  ";
  myCar1.model = "Swift";
  myCar1.year = 2019;

  // Put data into the second structure
  myCar2.brand = "Swift";
  myCar2.model = "Brezza";
  myCar2.year = 2021;

  // Print the structure members
  cout << myCar1.brand << " " << myCar1.model << " " << myCar1.year << "\n";
  cout << myCar2.brand << " " << myCar2.model << " " << myCar2.year << "\n";
  return 0;
}
```

Pointers

- A pointer is a variable that contains the address of another variable.
- It can be dereferenced with the help of (*) operator to access the memory location to which the pointer points.
- The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.
- The symbol of an address is represented by a pointer. In addition to creating and modifying dynamic data structures, they allow programs to emulate call-by-reference.
- One of the principal applications of pointers is iterating through the components of arrays or other data structures.
- The pointer variable that refers to the same data type as the variable you're dealing with has the address of that variable set to it (such as an int or string).

```
#include <iostream>
using namespace std;
int main()
{
    int number=30;
    int *   p;
    p=&number;       //stores the address of number variable
       number
    cout<<"Address of number variable is:"<<&number<<endl;
    cout<<"Address of p pointer variable is:"<<p<<endl;
    cout<<"Value of p variable is:"<<*p<<endl;
  return 0;
}
```

Reference

- A reference is a variable that is referred to as another name for an already existing variable.
- The reference of a variable is created by storing the address of another variable.
- A reference variable can be considered as a constant pointer with automatic indirection.
- Here, automatic indirection means that the compiler automatically applies the indirection operator (*).
- Reference can be created by simply using an ampersand (&) operator. When we create a variable, then it occupies some memory location.
- We can create a reference of the variable; therefore, we can access the original variable by using either name of the variable or reference. For example,

```
int a=10;
```

- Now, we create the reference variable of the above variable.

```
int &ref=a;
```

- The above statement means that 'ref' is a reference variable of 'a', i.e., we can use the 'ref' variable in place of 'a' variable.
- C++ provides two types of references:
  - References to non-const values
  - References as aliases

References to non-const values

It can be declared by using & operator with the reference type variable

```
#include <iostream>
using namespace std;
int main()
{
int a=10;
int &value=a;
cout << value << endl;
return 0;
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
  string food = "Pizza";
  string &meal = food;

  cout << food << "\n";
  cout << meal << "\n";
  return 0;
}
```

References as aliases

References as aliases is another name of the variable which is being referenced.

```
#include <iostream>
using namespace std;
int main()
{
int a=70;          // variable initialization
int &b=a;          // 'b' reference to a.
int &c=a;          // 'c' reference to a.
cout << "Value of a is :" <<a<< endl;
cout << "Value of b is :" <<b<< endl;
cout << "Value of c is :" <<c<< endl;
return 0;
}
```

User-Defined Data Types:

- Abstract or User-Defined data types are defined by the user itself. Like, defining a class in C++ or a structure.
- C++ provides the following user-defined datatypes:
  - ❖ Class
  - ❖ Structure
  - ❖ Union
  - ❖ Enumeration
  - ❖ Typedef defined Datatype

Class

- In C++, class is a group of similar objects.
- It is a template from which objects are created.
- It can have data members and member function methods

```
class Student       // Class declration
{
  public:
     int id;                  //data member (also instance variable)
     string name;         //data member(also instance variable)
     void insert(int i, string n)      // member function
      {
         id = i;
         name = n;
      }
     void display()                  // member function
      {
         cout<<id<<"  "<<name<<endl;
      }
};
```

Enumeration (or enum)

- Enumeration (or enum) is a user defined data type in C.
- It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.

```
// An example program to demonstrate working  of enum in C
#include<stdio.h>
enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};

int main()
{
    enum week day;
    day = Wed;
    printf("%d",day);
    return 0;
}
```

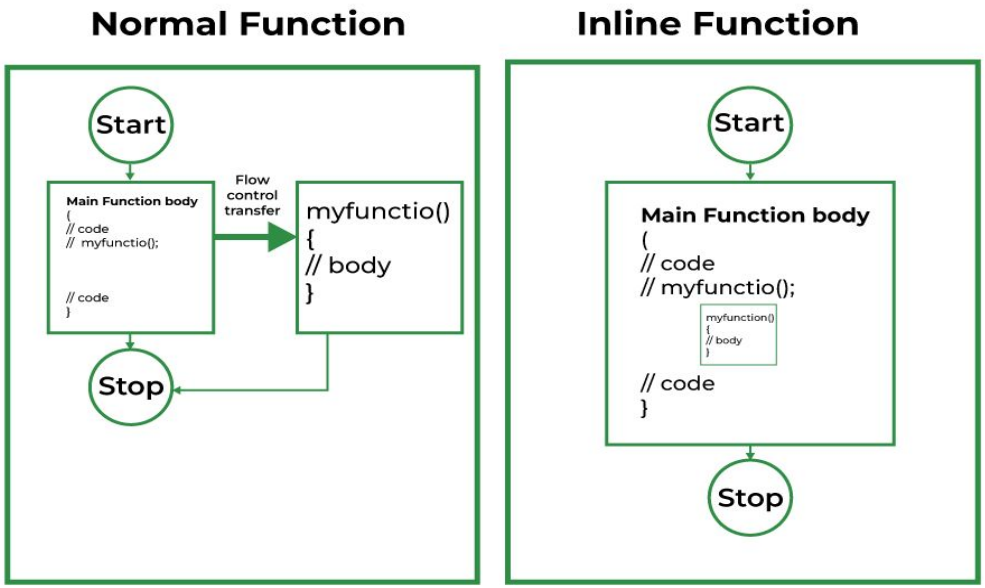
```
// Another example program to demonstrate working of enum in
    C ++
#include<stdio.h>

enum year{Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct,
    Nov, Dec};
int main()
{
int i;
for (i=Jan; i<=Dec; i++)
    cout<<i;
return 0;
}
```

Inline Functions in C++

- C++ provides inline functions to reduce the function call overhead.
- An inline function is a function that is *expanded in line* when it is called.
- When the inline function is called whole code of the inline function gets *inserted or substituted* at the point of the inline function call.
- This substitution is performed by the C++ compiler at *compile time*.
- An inline function may increase efficiency if it is small

Syntax

```
inline return-type function-name(parameters)
{
// function code
}
```



Why Inline Functions are Used?

- When the program executes the function call instruction the CPU stores the memory address of the instruction following the function call, copies the arguments of the function on the stack, and finally transfers control to the specified function.
- The CPU then executes the function code, stores the function return value in a predefined memory location/register, and returns control to the calling function.
- This can become overhead if the execution time of the function is less than the switching time from the caller function to called function (callee).
- For functions that are large and/or perform complex tasks, the overhead of the function call is usually insignificant compared to the amount of time the function takes to run.
- However, for small, commonly-used functions, the time needed to make the function call is often a lot more than the time needed to actually execute the function's code. This overhead occurs for small functions because the execution time of a small function is less than the switching time.

Inline functions Advantages:

- Function call overhead doesn't occur.
- It also saves the overhead of push/pop variables on the stack when a function is called.
- It also saves the overhead of a return call from a function.
- When you inline a function, you may enable the compiler to perform context-specific optimization on the body of the function. Such optimizations are not possible for normal function calls.
- Other optimizations can be obtained by considering the flows of the calling context and the called context.
- An inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function called preamble and return.

Inline function Disadvantages:

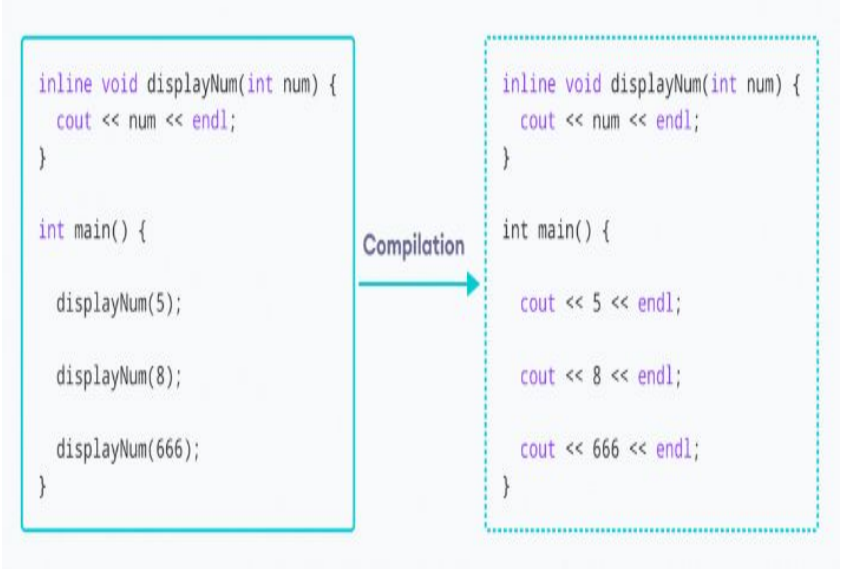
- If you use too many inline functions then the size of the binary executable file will be large, because of the duplication of the same code.
- Too much inlining can also reduce your instruction cache hit rate, thus reducing the speed of instruction fetch from that of cache memory to that of primary memory.
- The inline function may increase compile time overhead if someone changes the code inside the inline function then all the calling location has to be recompiled because the compiler would be required to replace all the code once again to reflect the changes, otherwise it will continue with old functionality.
- Inline functions may not be useful for many embedded systems. Because in embedded systems code size is more important than speed.
- Inline functions might cause thrashing because inlining might increase the size of the binary executable file. Thrashing in memory causes the performance of the computer to degrade. The following program demonstrates the use of the inline function

Program to demonstrate Inline Function

```
#include <iostream>
using namespace std;
inline int cube(int s)   // Function declaration with definition
{
  return s * s * s;
 }
int main()
{
    cout << "The cube of 3 is: " << cube(3) << "\n";
    return 0;
}
```

Example 2

```
#include <iostream>
using namespace std;
inline void displayNum(int num)
{
    cout << num << endl;
}

int main()
{
        displayNum(5);        // first function call
        displayNum(8);        // second function call
       displayNum(666);     // third function call
       return 0;
}
```



- In C++, we can declare a function as inline. This copies the function to the location of the function call in compile-time and may make the program execution faster.
- Here, we created an inline function named displayNum() that takes a single integer as a parameter.
- We then called the function 3 times in the main() function with different arguments. Each time displayNum() is called, the compiler copies the code of the function to that call location.

Friend function

- If a function is defined as a friend function in C++, then the *protected and private* data of a class can be accessed using the function.
- By using the keyword *friend* compiler knows the given function is a friend function.
- For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the *keyword friend*.

Declaration of friend function in C++

```
class class_name
{
    friend data_type function_name(argument/s);
        // syntax of friend function.
};
```

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part

A friend function tough not a member function has full rights and access to the members of the class

```
#include <iostream>
using namespace std;
        class demo
         {
         private:
         int a=5;
         friend void display(demo);   //friend function declaration
         };
                  void display(demo d)   //friend function defination
                 {
                  cout<<"a: "<<d.a<<endl;      //access of private data
                 }
         int main()
         {
         demo d;
         display(d);                          //calling a friend function
         return 0;
         }
```

when the function is friendly to two classes.

```
#include <iostream>
using namespace std;
class B;          // forward declarartion.
class A
{
   int x;
   public:
   void setdata(int i)
   {
      x=i;
   }
   friend void min(A,B);         // friend function.
};
class B
{
   int y;
   public:
   void setdata(int i)
   {
      y=i;
   }
   friend void min(A,B);                  // friend function
};
void min(A a,B b)
{
   if(a.x<=b.y)
   std::cout << a.x << std::endl;
   else
   std::cout << b.y << std::endl;
}
  int main()
{
  A a;
  B b;
  a.setdata(10);
  b.setdata(20);
  min(a,b);
   return 0;
 }
```

Example of a friend class.

```
#include <iostream>

using namespace std;

class A
{
   int x =5;
   friend class B;          // friend class.
};
class B
{
  public:
   void display(A &a)
   {
      cout<<"value of x is : "<<a.x;
   }
};
int main()
{
   A a;
   B b;
   b.display(a);
   return 0;
}
```

Advantages of Friend Functions

- **Versatility in Overloaded Operators:** Friend functions allow overloaded operators to access private members of a class, enhancing flexibility in defining custom behavior for operators like +, -, etc.
- **Interclass Interactions:** Friend facilitates seamless communication between different classes by granting access to private and protected members, promoting modularity and encapsulation.
- **Serialization: friend function C++** plays a vital role in serialization, enabling classes to efficiently serialize and deserialize their private data without compromising encapsulation.
- **Type Conversion:** Friend functions enable type conversion by accessing private members. This allows for smooth conversion between different data types, enhancing code flexibility and readability.
- **Domain-Specific Language:** Friend functions empower developers to create domain-specific languages within C++ programs, enhancing expressiveness and readability by enabling custom syntax and semantics.

Disadvantages of Friend Functions:

- **Limited Polymorphism:** Friend functions are not polymorphic, limiting their usage in scenarios requiring dynamic behavior based on the object's runtime type.
- **Inheritance:** Friend functions can't access private members of derived classes, potentially complicating inheritance hierarchies and violating the principle of least privilege.
- **Code Complexity:** Excessive Use of friend functions can lead to increased code complexity and reduced maintainability due to tight coupling between classes and functions. It can also hinder code comprehension and debugging.

Constructor

- Constructor is a special member function of a class that is used to initialize the objects of the class.
- Its called special because its name, same as class name
- Constructors do not have any return type and are automatically called when the object is created.

Characteristics of constructors

- The name of the constructor is the same as the class name
- No return type is there for constructors
- Called automatically when the object is created
- Always placed in the public scope of the class
- If a constructor is not created, the default constructor is created automatically and initializes the data member as zero
- Declaration of constructor name is case-sensitive
- A constructor is not implicitly inherited

Types of constructors

- **Default constructor -** A default constructor is one that does not have function parameters. It is used to initialize data members with a value. The default constructor is called when the object is created.
- **Parameterized constructor -** A non-parameterized constructor does have the constructor arguments and the value passed in the argument is initialized to its data members. Parameterized constructors are used in constructor overloading.
- **Copy constructor -** Copy constructor initializes an object using another object of the same class.
- **Constructor overloading in C++**
- As there is a concept of function overloading, similarly constructor overloading is applied. When we overload a constructor more than a purpose it is called constructor overloading.
- The declaration is the same as the class name but as they are constructors, there is no return type.
- The criteria to overload a constructor is to differ the number of arguments or the type of arguments.

Default constructor -

```
#include <iostream>
using namespace std;
class construct                         // create a class construct
{
public:
   int a, b;                        // initialise two data members
   construct()                                  // this is how default constructor is created
   {
      a = 10;                  // intialise  a with some value
      b = 20;                 // initialise  b with some value
   }
};
int main()
{
   construct c;              // creating an object of construct calls defualt constructor
   cout<< "a:" <<c.a<<endl   << "b:" <<c.b; // print a and b
   return 1;
}
```

// Example of Defining the default Constructor Within the Class

```
#include <iostream>
using namespace std;
class Car {
public:
    string brand;
    string model;
    int year;

    Car(  )                    // Constructor defined within the class
          {
        brand = “Maruti";
        model = “800";
        year = 1990;
          }
};

int main() {
    Car myCar;
    cout << "Brand: " << myCar.brand << endl;
    cout << "Model: " << myCar.model << endl;
    cout << "Year: " << myCar.year << endl;
    return 0;
}
```

Explanation

- In this example, we have a class Car with member variables brand, model, and year.
- The constructor Car() is defined within the class itself.
- Inside the constructor, default values for brand, model, and year are assigned to “maruti”, “800”, and 1990 respectively.
- When an object myCar of the Car class is created in the main() function, the constructor Car() is automatically called, initializing the object with default values.
- The program then prints out the brand, model, and year of myCar, which are set to “maruti”, “800”, and 1990 respectively.

Real World Example Defining the Constructor Outside the Class

```
#include <iostream>
using namespace std;
class Car {
public:
    string brand;
    string model;
    int year;
    Car(); // Constructor declaration
};
// Constructor definition outside the class
Car::Car()
{
    brand = "Ford";
    model = "Mustang";
    year = 2020;
}

int main() {
    Car myCar;
    cout << "Brand: " << myCar.brand << endl;
    cout << "Model: " << myCar.model << endl;
    cout << "Year: " << myCar.year << endl;
    return 0;
}
```

Explaination

- Here, we declare the constructor Car() within the class Car without defining it.
- Outside the class, we define the constructor Car::Car() which initializes the brand, model, and year member variables of the Car class with the values “Ford”, “Mustang”, and 2020 respectively.
- When myCar object is created in the main() function, the constructor Car::Car() is automatically invoked to initialize it.
- The program then prints out the brand, model, and year of myCar, which are set to “Ford”, “Mustang”, and 2020 respectively.

```
#include <iostream>
using namespace std;

class Person {
public:
    string name;
    int age;

    // Default constructor definition
    Person() {
        name = “Rakesh ";
        age = 35;
    }
};

int main() {
    Person personObj;
    cout << "Name: " << personObj.name << endl;
    cout << "Age: " << personObj.age << endl;
    return 0;
}
```

Parameterized Constructor

- A parameterized constructor has parameters that allow the initialization of member variables with specific values passed during object creation.
- **How To Create A Parameterized Constructor**?
- To create a parameterized constructor, you define a constructor within the class with parameters corresponding to the values you want to initialize.
- Syntax

```
class ClassName {
public:
    ClassName(Type1 parameter1, Type2 parameter2, ...);
                        // Parameterized constructor declaration
};
```

```
#include <iostream>
using namespace std;
class Rectangle {
private:
    int length;
    int width;
public:
    Rectangle(int l, int w)                    // Parameterized constructor defined inside the class
 {
        length = l;
        width = w;
    }
    int area() {
        return length * width;
    }
};

int main() {
    Rectangle rect(5, 3); // Creating object with parameters
    cout << "Area of Rectangle: " << rect.area() << endl;
    return 0;
}
```

Explanation

- In this example, we have a class Rectangle with member variables length and width.
- The parameterized constructor Rectangle(int l, int w) is defined within the class.
- It initializes the length and width of member variables with the values passed as parameters.
- When an object rect of the Rectangle class is created in the main() function with parameters 5 and 3, the parameterized constructor is automatically called, initializing the object with specified values.
- The program then calculates and prints the area of the rectangle, which is 15.

Copy Constructor In C++

- A copy constructor in C++ is a special member function that initializes a new object as a copy of an existing object of the same class.
- It is called when an object is passed by value, returned by value, or initialized with another object of the same class.
- Syntax

```
class ClassName {
public:
    ClassName(const ClassName& obj);
        // Copy constructor declaration
};
```

```
#include <iostream>
using namespace std;

class Person {
public:
    string name;

    // Explicit copy constructor with parameterized constructor
    Person(const Person& obj) {
        name = obj.name;
    }

    Person(string n) {
        name = n;
    }
};

int main() {
    Person person1("Charlie");
    Person person2(person1); // Copying person1 to person2 using parameterized constructor
    cout << "Name of person2: " << person2.name << endl;
    return 0;
}
```

Question :

- Explain different feature of OPP's ?

  Ans : - object,classes,inheritance,polymorishm, data abstraction,encapsulation etc
- Give difference between POP's and OOP's ?
- Explain different data type supported by C+
- What is constructor ? Explain its type with example.
- What is inline function ? Explain its with example.
- What is friend function ? ? Explain its with example.

- UNI II